

--zsh

...

...ory_and_test_shell.c

...ano signal_handler.c

--zsh

-- append_stderr

UW PICO 5.09

File: history_pipeline.c

```
    free(hist.entries[hist.start]);
    hist.entries[hist.start] = strdup(cmd);
    hist.start = (hist.start + 1) % MAX_HIST;
} else {
    int idx = (hist.start + hist.count) % MAX_HIST;
    hist.entries[idx] = strdup(cmd);
    hist.count++;
}
hist.next_id++;
}

void display_history() {
    printf("--- Command History (Max Capacity: %d) ---\n", MAX_HIST);
    for (int i = 0; i < hist.count; i++) {
        int idx = (hist.start + i) % MAX_HIST;
        int display_id = hist.next_id - hist.count + i;
        printf(" %d %s\n", display_id, hist.entries[idx]);
    }
}

void cleanup_history() {
    for (int i = 0; i < hist.count; i++) {
        int idx = (hist.start + i) % MAX_HIST;
        free(hist.entries[idx]);
    }
    hist.count = 0;
}

void execute_pipeline(char *cmd_str) {
    char *cmds[MAX_CMDS];
    int num_cmds = 0;

    char *token = strtok(cmd_str, "|");
    while (token != NULL && num_cmds < MAX_CMDS) {
        cmds[num_cmds++] = token;
        token = strtok(NULL, "|");
    }
    if (num_cmds == 0) return;

    int pipefds[2 * (num_cmds - 1)];
    for (int i = 0; i < num_cmds - 1; i++) {
        if (pipe(pipefds + i * 2) < 0) {
            perror("Pipe creation failed");
            return;
        }
    }

    for (int i = 0; i < num_cmds; i++) {
```

 Get Help
 Exit

 WriteOut
 Justify

 Read File
 Where is

 Prev Pg
 Next Pg

 Cut Text
 UnCut Tex

UW PICO 5.09

File: history_pipeline.c

```

    }
    hist.count = 0;
}

void execute_pipeline(char *cmd_str) {
    char *cmds[MAX_CMDS];
    int num_cmds = 0;

    char *token = strtok(cmd_str, "|");
    while (token != NULL && num_cmds < MAX_CMDS) {
        cmds[num_cmds++] = token;
        token = strtok(NULL, "|");
    }
    if (num_cmds == 0) return;

    int pipefds[2 * (num_cmds - 1)];
    for (int i = 0; i < num_cmds - 1; i++) {
        if (pipe(pipefds + i * 2) < 0) {
            perror("Pipe creation failed");
            return;
        }
    }

    for (int i = 0; i < num_cmds; i++) {
        char *args[MAX_ARGS];
        int arg_count = 0;
        char *arg_token = strtok(cmds[i], " \t");
        while (arg_token != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = arg_token;
            arg_token = strtok(NULL, " \t");
        }
        args[arg_count] = NULL;

        if (arg_count == 0) continue;

        pid_t pid = fork();
        if (pid == 0) {
            if (i != 0) {
                dup2(pipefds[(i - 1) * 2], STDIN_FILENO);
            }
            if (i != num_cmds - 1) {
                dup2(pipefds[i * 2 + 1], STDOUT_FILENO);
            }

            for (int j = 0; j < 2 * (num_cmds - 1); j++) {
                close(pipefds[j]);
            }

```

UW PICO 5.09

File: history_pipeline.c

```

        execvp(args[0], args);
        perror("Execution failed");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < 2 * (num_cmds - 1); i++) {
        close(pipefds[i]);
    }

    for (int i = 0; i < num_cmds; i++) {
        wait(NULL);
    }
}

int main() {
    char input_buf[BUF_SIZE];
    char input_copy[BUF_SIZE];

    while (1) {
        printf("\nPipeline Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;
        input_buf[strcspn(input_buf, "\n")] = 0;
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        strcpy(input_copy, input_buf);
        add_history(input_copy);

        if (strcmp(input_buf, "history") == 0) {
            display_history();
        } else {
            execute_pipeline(input_buf);
        }
    }

    cleanup_history();
    printf("Shell exited safely.\n");
    return 0;
}

```

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano history_pipeline.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall history_pipeline.c -o history_pipeline
./history_pipeline
```

```
Pipeline Shell > ls -l | grep history
-rwxr-xr-x  1 medikondarakesh  staff   51664  2 Oct 14:28 history_pipeline
-rw-r--r--  1 medikondarakesh  staff    3411  2 Oct 14:28 history_pipeline.c
drwxr-xr-x  3 medikondarakesh  staff      96  2 Oct 14:28 history_pipeline.dSYM
-rwxr-xr-x  1 medikondarakesh  staff   34832  1 Oct 09:02 shell_history
-rw-r--r--  1 medikondarakesh  staff    3323  1 Oct 09:09 shell_history.c
drwxr-xr-x  3 medikondarakesh  staff      96  1 Oct 09:02 shell_history.dSYM
```

```
Pipeline Shell > cat /etc/passwd | cut -d: -f1 | sort
_accessoryupdater
_amavisd
_analyticsd
_aonsensed
_appinstalld
_appleevents
_applepay
_appowner
_appserver
_appstore
```

~ — -zsh

...

...ory_and_test_shell.c

...ano signal_handler.c

~ — -zsh

~ — append_stderr

```
_postfix
_postgres
_qtss
_reportmemoryexception
_reportsystemmemory
_rmd
_sandbox
_screensaver
_scsd
_securityagent
_sntpd
_softwareupdate
_spinadd
_spotlight
_sshd
_svn
_swtransparencyd
_systemstatustd
_taskgated
_teamsserver
_terminusd
_timed
_timezone
_tokend
_trustd
_trustevaluationagent
_unknown
_update_sharing
_usbmuxd
_uucp
_warmd
_webauthserver
_windowserver
_www
_wwwproxy
_xserverdocs
#
#
# in single-user mode. At other times this information is provided by
# Note that this file is consulted directly only when the system is running
# Open Directory.
# Open Directory.
# See the opendirectoryd(8) man page for additional information about
# User Database
##
##
daemon
nobody
root
```

```
Pipeline Shell > exit
```

```
Shell exited safely.
```

```
medikondarakesh@Medikondas-MacBook-Air ~ % █
```